Prototype Retrospective & Lessons Learned

Two-tier control plane for 6G network slicing (fast ML + LLM/MCP compiler). Everything found from day one: what we built, what we chose, what went wrong, what went right, and what can be better.

1. What this is

A research prototype for a master's thesis on 6G network slicing. Tier 1 runs fast ML (a load forecaster + a numpy MLP KPI predictor) and an analytic capacity model; Tier 2 is an LLM (Ollama cloud) that decides per-slice replica counts and acts through typed MCP tools; a verification gate checks every change; an offline hindsight oracle scores every decision against future load.

It is a **simulation**, not a live cluster. No kubectl, no pods, no real KPI telemetry — delay/loss/violation are always recomputed from the analytic capacity model, and the dataset supplies only the per-slice load.

2. Components

• **Capacity model** (analytic, not ML) — the shared ground truth for env, oracle, gate, baselines and MCP tools:

```
effective_capacity = replicas * cap
rho = load / capacity
delay = base / (1 - rho)  if rho < 1 else max_delay
loss  = (load - capacity) / load if overloaded else 0
violation = delay > budget OR loss > budget
required_replicas = ceil(load / (cap * 0.7))
```

• **NFConfig per slice**: URLLC tight delay, eMBB loose, mMTC strict loss.

• **Forecaster**: linear regression over the last 5 aggregate-load points (or persistence).

• **KPI predictor**: numpy MLP (16 hidden, ReLU, MSE) on aggregate load → delay; p_violation = sigmoid((pred_delay – budget)/budget).

• **Baselines**: threshold (reactive), hpa (ideal), predict-then-threshold (degenerates to hpa), capacity-rule (deterministic rule).

• **MCP server**: six typed tools — estimate_capacity, scale_replicas, set_resources, add_nf_instance, apply_k8s_manifest, inspect_infrastructure.

• **LLM compiler**: OpenAI-compatible tool-calling against Ollama localhost; tool-call-as-answer (no JSON parsing).

• **Gate + oracle**: verification gate (schema + policy, one retry); hindsight oracle (greedy future-load P95, offline only).

3. What we did, in order

• Interviewed the plan → scope locked: **prototype first**, 1 month, commercial-API LLM.

• Investigated datasets → excluded OpenRAN (too large); kept the Farreras et al. 2024 slicing dataset (Zenodo 10610616) as the only source.

• Considered feeding a DeepSlice-style ML model into the LLM; settled on forecaster + KPI predictor + analytic capacity model as Tier 1.

• Corrected PLAN.md, drew the PlantUML architecture, wrote the papers reference matrix.

• Built the end-to-end prototype: dataset → env → capacity model → oracle → gate → MCP → LLM → baselines → benchmark → reports.

• Generated initial_results.pdf, then expanded analysis.pdf.

• Did EDA as a Jupyter notebook (pandas head + manual pipeline walk) and a step-by-step LLM harness notebook.

• Refactored the LLM from JSON output to **tool-calling** (scale_replicas calls ARE the answer).

• Renamed llm-stub → **capacity-rule** (it is a deterministic rule, not an LLM).

• Added the inspect_infrastructure tool and showed (in isolation) it makes the LLM near-exact.

• Ran RQ5 (estimate_capacity A/B), tool-call traces, and a prompt A/B (reverted).

• Excluded **mistral-large-4** (too expensive, same decisions).

• Ran the full closed-loop LLM benchmark (3 models × ramp/steady/spike) for the final results.

• Merged the two reports into a single analysis_results.pdf and wrote this retrospective.

4. Key choices and why

• **Two tiers, LLM outside the fast loop**: ML gives fast numeric signals; the LLM is called only for scaling/reconfig — opposite of putting the LLM in the RRM loop.

• **Analytic capacity model as ground truth**: deterministic, reusable by env, oracle, gate, baselines and tools. Removes ambiguity about what is 'correct'.

• **Tool-calling instead of JSON**: no brittle parsing; the LLM's scale_replicas calls are the action, and the MCP layer clamps/validates at call time.

• **inspect_infrastructure as the primary tool**: it surfaces utilization, headroom, required replicas and budgets per slice, so the LLM defers to the model.

• **Verification gate + offline hindsight oracle**: gate = runtime safety; oracle = offline scoring against future load. Kept separate so the oracle never leaks into the system.

• **Baselines ordered from dumb to ideal**: threshold → hpa → predict-then-threshold → capacity-rule, so the LLM is judged against a ladder, not a single strawman.

• **Ollama cloud models over localhost**: no token, OpenAI-compatible; 3 models kept, mistral dropped.

• **numpy + stdlib core** (no sklearn/pandas/gym in src/); pandas/jupyter/matplotlib/reportlab only in a .venv for notebooks and PDFs.

5. Final results (per-slice closed loop)

Ramp, 100 states. llm-* = tool-calling LLM (inspect_infrastructure + scale_replicas).

Caveat: single run, no repetitions. Baselines are deterministic; each LLM result is one trajectory per model per scenario — no error bars, variance not measured.

method	viol%	repl	under	over	mae
threshold	24.67	2.50	1.11	0.11	0.49
hpa	15.67	2.63	0.95	0.08	0.00
predict-then-threshold	15.67	2.63	0.95	0.08	0.00
capacity-rule	1.00	3.73	0.16	0.38	1.10
llm-flash	10.00	3.09	0.74	0.33	0.47
llm-glm5.3	8.00	3.50	0.57	0.57	0.87
llm-deepseek	13.33	2.83	0.86	0.19	0.21

steady (80 states) and spike (80 states) hold the same ordering: capacity-rule lowest violation (~0.8–1.7%), then llm-glm5.3 / llm-flash, then hpa/threshold. Full tables in analysis_results.pdf.

5.1 Research questions answered

• **RQ1 (valid, safe changes)**: yes within tested scenarios — gate enforces bounds, tool_trace.json confirms correct calls, rejections proven in unit tests. Claim 'measured rejection rates', never '100% safe'.

• **RQ2 (under/over vs oracle)**: closed-loop, flash and glm-5.3 over-provision (over 0.31–0.85), deepseek under-provisions (under 0.81–1.11). The inspect tool does NOT collapse the closed-loop error the way it did

in isolated A/B.

• **RQ3 (latency/cost)**: flash 2.8s < glm-5.3 4.6s < deepseek 7.2s; mistral ~2× tokens and excluded.

• **RQ4 (YAML vs MCP)**: both wired, not head-to-head. MCP typed tools are safer by construction (bounds at call time).

• **RQ5 (estimate_capacity)**: cuts MAE 1.22→0.22 and cost 3.53→2.57 but raises violation 3.3%→10.0% (exact, not safer). inspect_infrastructure is the better tool.

• **RQ6 (LLM vs rules)**: tool-calling LLM beats threshold/hpa (5.4–16.7 vs 14.6–28.4%) but trails capacity-rule (~1%).

6. capacity-rule: an interesting baseline, but too expensive

capacity-rule is the most interesting non-LLM baseline — it reaches ~1% violation with a trivial deterministic rule. But it does so by **over-provisioning**: it holds the highest mean replicas of any method (3.73–4.28 vs hpa 2.24–2.63 and the LLM 2.62–3.50).

It is **not** prescient — it sees the same payload as the LLM (current load + forecast + p_violation) and shares the oracle's required_replicas *formula*, not its future *data*. Its edge is hysteresis: it scales up on forecast violation but only scales down one replica at a time, so it ratchets up and stays high, buying safety with idle capacity.

So it is a useful **upper bound** on safety (the ceiling the model allows), but too expensive to be a practical controller — it wastes compute to buy low violation. Keep it as the reference, not as a competitor to beat.

7. What went well

• End-to-end loop closed early: dataset → env → model → oracle → gate → MCP → LLM → benchmark → report, with 8 passing pytest tests.

• Tool-call-as-answer removed the whole JSON-parsing fragility; the action is read straight from the last scale_replicas call per slice.

• One analytic capacity model reused everywhere eliminated inconsistency and made every part comparable.

• inspect_infrastructure is a genuinely good tool: in isolation it drops every model's MAE to ≈0 by handing the LLM the model's answer.

• Resumable experiment scripts meant rate-limit crashes only lost the current model, not the whole run.

• The 'tool, not the model, wins' finding (from isolation) is a real, honest result worth keeping in the paper — with the caveat that the closed loop is different.

8. What went wrong (and the fix)

• **Constant load**: maxAvgLambda is fixed (18 Mbps), so aggregate load looked flat — switched to per-slice bandwidth from the slices JSON as the load signal.

• **numpy truthiness**: `if delays` on an array raised ValueError — changed to `if len(delays)`.

• **reportlab name collision**: `cm` (capacity_model alias) clashed with reportlab's `cm` unit — renamed the import to `cmod`.

• **matplotlib API change**: 3.11 removed `boxplot(labels=)` — changed to `tick_labels=`.

• **Notebook cwd**: running from notebooks/ broke the relative dataset path — added sys.path + os.chdir bootstrap in the first cell.

• **Notebook JSONDecodeError**: single-round reconstruction assumed a final JSON — made the harness loop robust over rounds.

• **inspect_infrastructure string-state bug**: a model passed the state as a double-encoded JSON string and got zeros back — added json.loads + 'slices' unwrap.

• **429 rate limits / timeouts**: hit repeatedly (once lost credits mid-run) — added retry with backoff in chat(), then in _post(); added 5s spacing between decisions.

• **pkill self-match**: a pkill pattern matched the running shell and killed it (exit 144) — avoided patterns that match the command line.

• **PDF prompt clipping**: a preformatted prompt got cut off — switched to Paragraph with </> escaping.

• **architecture.png stretched**: 4.86:1 forced to 2:1 — fixed to width=16cm, height=3.3cm.

• **env off-by-one**: loop ran one step too many — tests adjusted to 3 steps.

• **Resumability skipped stale data**: inspect_scenarios and tool_trace skipped everything because old results existed — deleted the stale JSONs and re-ran fresh.

• **_post had no retry**: a single 180s timeout killed the whole benchmark — added 6× retry with 60s backoff (mirroring chat()).

• **mistral-large-4**: ~3× slower, ~2× tokens, identical decisions — excluded as pure cost.

9. What can be better (future work)

• **Real cluster, not simulation**: run the gate against kind + Open5GS so the K8s change is applied and the KPI is measured, not recomputed.

• **Mode T vs Mode D head-to-head**: the plan defined both; only the combined system was tested. Separate fidelity (faithful compile) from decision quality.

• **YAML vs MCP head-to-head**: both are wired but never compared quantitatively.

• **Fix or drop predict-then-threshold**: it degenerates to hpa (the MLP over-predicts delay). Either retrain the MLP or replace the baseline.

• **Repetitions / variance**: only one run per model; temp 0 helps but is not bit-identical. Report N runs + std.

• **Perturb the oracle**: oracle and estimate_capacity share the same model (circularity). Perturb one to prove the result is not trivial.

• **Small local LLM**: RQ3 asked local vs commercial, but only cloud models ran. Add a 3–7B local model.

• **Ablations**: raw numbers vs qualitative labels, with/without uncertainty, with/without current cluster state — none of these were run.

• **Closed-loop vs isolated mismatch**: state explicitly that the inspect tool helps in isolation but does not close the closed-loop gap, since the oracle scores future load.

10. Where everything lives

• analysis_results.pdf — final merged report.

• PLAN.md — plan + findings (§15).

• docs/findings.md — consolidated state.

• docs/writing-the-paper.md — paper process.

• results/*.json — raw data.

• papers/reference_matrix.md — related work.

• notebooks/ — EDA + LLM harness.